

实验报告

系统开发工具基础 第 3 周

姓 名： 王本熙

学 号： 25020007117

日 期： 2026 年 9 月 7 日

平 台： Windows + WSL (Ubuntu)

目录

1 实验目的	3
2 实验环境	3
3 q09: 从源码构建并在干净环境安装 Wheel	3
3.1 题目要求	3
3.2 实验步骤	4
3.3 运行结果与截图	5
3.4 结果分析	5
4 q10: 让编程智能体进入可验证的修复循环	5
4.1 题目要求	5
4.2 实验步骤	7
4.3 运行结果与截图	8
4.4 结果分析	8
5 q11: 把”无法处理”的协作材料改成可执行信息	8
5.1 题目要求	8
5.2 实验步骤	8
5.3 运行结果与截图	10
5.4 结果分析	11
6 q12: 修复一个可复现的线性回归训练循环	11
6.1 题目要求	11
6.2 实验步骤	11
6.3 运行结果与截图	12
6.4 结果分析	13
7 第 3 周课后练习	13
7.1 虚拟环境激活前后发生了什么	13
7.2 创建带锁文件的 Python 包并安装	13
7.3 Docker + docker compose 搭建课程网站	14
7.4 Python 应用 + Redis 的 Dockerfile 与 docker-compose.yml	14
7.5 发布 Python 包到 TestPyPI 并推送 Docker 镜像到 ghcr.io	14
7.6 GitHub Pages 搭建网站	14
7.7 同一任务四种方式对比: LeetCode Two Sum	14
7.8 用 AI 智能体浏览陌生代码库	15
7.9 从零开始氛围编程一个小应用	15
7.10 AGENTS.md / Skill / 子智能体	15

7.11 Issue 评估: jupyterlab #15842	15
7.12 最小可复现示例练习	16
8 代码与仓库	16
8.1 Git 提交记录	16
9 结论	17

第 1 章 实验目的

本实验旨在掌握以下内容：

- 从源码构建 Python 命令行包并在干净虚拟环境中通过 wheel 安装（q09）；
- 让编程智能体进入可验证的测试驱动修复循环：先写失败测试，再让智能体修改实现并通过（q10）；
- 将”无法处理”的协作材料改写为可执行信息：Issue、提交信息、评审意见的专业写法（q11）；
- 补全 PyTorch 线性回归训练循环：zero_grad/backward/step、eval/no_grad 评估模式，最终损失应小于 0.001（q12）。

第 2 章 实验环境

表 1: 实验环境

项目	配置
操作系统	Windows + WSL (Ubuntu 24.04)
Shell	bash
Python	3.12.3
编辑器	VS Code / Trae IDE / micro
版本管理	Git
远程仓库	GitHub
PyTorch	2.1.0+cpu

第 3 章 q09：从源码构建并在干净环境安装 Wheel

3.1 题目要求

主题：Packaging and Shipping Code。创建一个最小 Python 命令行包，并验证 wheel 安装：

- 创建 `src/greetlab/__init__.py`、给定的 `cli.py` 和 `pyproject.toml`，把”学号”替换为自己的学号 25020007117；
- 在已安装 build 工具的课程环境中运行 `python -m build`，生成 wheel；
- 新建一个干净虚拟环境，只从生成的 wheel 安装；
- 切换到 q09 之外运行 `sdt-greet --name < 学号 >`，确认输出正确。

3.2 实验步骤

按题目给定内容创建三个文件（pyproject.toml 中学号已替换为 25020007117）：

```
1 # src/greetlab/__init__.py
2 # 空文件，标记目录为 Python 包
3
4 # src/greetlab/cli.py
5 import argparse
6
7 def main():
8     p = argparse.ArgumentParser()
9     p.add_argument("--name", required=True)
10    a = p.parse_args()
11    print(f"Hello, {a.name}!")
```

```
1 [build-system]
2 requires = ["setuptools>=68"]
3 build-backend = "setuptools.build_meta"
4
5 [project]
6 name = "greetlab-25020007117"
7 version = "0.1.0"
8
9 [project.scripts]
10 sdt-greet = "greetlab.cli:main"
11
12 [tool.setuptools.packages.find]
13 where = ["src"]
```

随后在 WSL 终端中依次执行：

```
1 mkdir -p q09/src/greetlab
2 micro q09/src/greetlab/__init__.py # 空文件
3 micro q09/src/greetlab/cli.py      # 写入 main()
4 micro q09/pyproject.toml           # 替换学号
5
6 cd q09
7 python3 -m venv venv
8 source venv/bin/activate
9 pip install build -i https://pypi.tuna.tsinghua.edu.cn/simple
10
11 python -m build # 生成 dist/greetlab_25020007117-0.1.0-py3-none-any.whl
```

构建成功后，创建干净虚拟环境验证 wheel 安装：

```
1 cd ~
2 python3 -m venv venv_clean
```

```
3 source venv_clean/bin/activate
4 pip install ~/q09/dist/greetlab_25020007117-0.1.0-py3-none-any.whl
5
6 sdt-greet --name 25020007117 # 输出: Hello, 25020007117!
```

3.3 运行结果与截图

图 1 记录了从创建包结构、`python -m build` 构建到干净环境验证的全过程。

3.4 结果分析

- **包结构与 `pyproject.toml`:** 使用 `src-layout` (`src/greetlab/`), `pyproject.toml` 配置 `[project.scripts]` 定义入口点 `sdt-greet = "greetlab.cli:main"`, `[tool.setuptools.packages.find]` 指定 `where = ["src"]`, 保证构建时能正确找到包;
- **`python -m build`:** 该工具自动创建隔离的构建环境, 依次运行 `sdist` 和 `bdist_wheel`, 生成 `dist/` 目录下的 `.tar.gz` 和 `.whl`; 日志中 `Successfully built greetlab_25020007117-0.1.0-py3-n` 确认构建成功;
- **干净环境验证:** `venv_clean` 不依赖源码、不安装 `pytest/ruff`, 仅从 `wheel` 安装包。运行 `sdt-greet --name 25020007117` 输出 `Hello, 25020007117!`, 证明 `wheel` 自包含了所有运行时所需内容, 入口点正确注册到 `PATH`;
- **无 `README` 警告:** 构建日志中 `warning: sdist: file not found: README, README.rst, README.txt, README.md` 仅为警告, 不影响 `wheel` 生成——正式项目应在 `pyproject.toml` 中配置 `[project.readme]` 消除该警告。

第 4 章 q10: 让编程智能体进入可验证的修复循环

4.1 题目要求

主题: 智能体编程。复用第 9 题的 `greetlab` 包, 使用编程智能体完成一个小型测试驱动修复:

- 复制 q09 为 q10, 添加一个失败测试: 当 `name` 只含空白字符时, `main` 应以 `SystemExit(2)` 结束;
- 先运行测试确认失败, 再向编程智能体说明目标、约束和测试命令, 要求它修改实现并运行测试;
- 人工检查 `diff`, 撤销无关修改, 并再次运行测试;
- 在 `ai_log.md` 中用不超过 5 行记录核心提示、智能体改动和人工验证。

```
wangbenxi@WIN-S6NAAMF7V: x Ubuntu:24.04
writing manifest file 'greetlab_25020007117.egg-info/SOURCES.txt'
* Installed build dependency versions:
- setuptools==84.0.0
* Building wheel...
running bdist_wheel
running build
running build_py
creating build/lib/greetlab
copying src/greetlab/cli.py -> build/lib/greetlab
copying src/greetlab/_init_.py -> build/lib/greetlab
running egg_info
writing greetlab_25020007117.egg-info/PKG-INFO
writing dependency links to greetlab_25020007117.egg-info/dependency_links.txt
writing entry points to greetlab_25020007117.egg-info/entry_points.txt
writing top-level names to greetlab_25020007117.egg-info/top_level.txt
reading manifest file 'greetlab_25020007117.egg-info/SOURCES.txt'
writing manifest file 'greetlab_25020007117.egg-info/SOURCES.txt'
installing to build/bdist.linux-x86_64/wheel
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel./greetlab
copying build/lib/greetlab/_init_.py -> build/bdist.linux-x86_64/wheel./greetlab
running install_egg_info
Copying greetlab_25020007117.egg-info to build/bdist.linux-x86_64/wheel./greetlab_25020007117-0.1.0-py3.12.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_25020007117-0.1.0.dist-info/WHEEL
creating /home/wangbenxi/q09/dist/.tmp-Lxk408rs/greetlab_25020007117-0.1.0-py3-none-any.whl and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/_init_.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007117-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007117-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007117-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007117-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007117-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_25020007117-0.1.0-py3-none-any.whl
greetlab_25020007117-0.1.0-py3-none-any.whl
(wenv) wangbenxi@WIN-S6NAAMF7V: q09$ cd ~
source venv/bin/activate
sdt-greet --name 25020007117
Hello, 25020007117!
(wenv_clean) wangbenxi@WIN-S6NAAMF7V: q09$
```

(a) 创建 src/greetlab/ 包结构、pyproject.toml、venv、pip install build、python -m build 构建 sdist 与 wheel

```
wangbenxi@WIN-S6NAAMF7V: q09$ python3 --version
Python 3.12.3
wangbenxi@WIN-S6NAAMF7V: q09$ mkdir q09
cd q09
mkdir: cannot create directory 'q09': File exists
wangbenxi@WIN-S6NAAMF7V: q09$ mkdir -p src/greetlab
wangbenxi@WIN-S6NAAMF7V: q09$ micro src/greetlab/_init_.py
wangbenxi@WIN-S6NAAMF7V: q09$ micro src/greetlab/_init_.py
wangbenxi@WIN-S6NAAMF7V: q09$ micro src/greetlab/cli.py
wangbenxi@WIN-S6NAAMF7V: q09$ micro pyproject.toml
wangbenxi@WIN-S6NAAMF7V: q09$ python3 -m venv venv
source venv/bin/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install build
python -m build
writing to /home/wangbenxi/.config/pip/pip.conf
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: build in ./venv/lib/python3.12/site-packages (1.6.1)
Requirement already satisfied: setuptools<24.0 in ./venv/lib/python3.12/site-packages (from build) (26.3)
Requirement already satisfied: pyproject_hooks in ./venv/lib/python3.12/site-packages (from build) (1.2.0)
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
- setuptools==68
* Getting build dependencies for sdist...
running egg_info
creating src/greetlab_25020007117.egg-info/PKG-INFO
writing dependency links to src/greetlab_25020007117.egg-info/dependency_links.txt
writing entry points to src/greetlab_25020007117.egg-info/entry_points.txt
writing top-level names to src/greetlab_25020007117.egg-info/top_level.txt
writing manifest file 'src/greetlab_25020007117.egg-info/SOURCES.txt'
reading manifest file 'src/greetlab_25020007117.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007117.egg-info/SOURCES.txt'
* Installed build dependency versions:
- setuptools==84.0.0
* Building sdist...
running sdist
running egg_info
writing src/greetlab_25020007117.egg-info/PKG-INFO
writing dependency links to src/greetlab_25020007117.egg-info/dependency_links.txt
writing entry points to src/greetlab_25020007117.egg-info/entry_points.txt
writing top-level names to src/greetlab_25020007117.egg-info/top_level.txt
reading manifest file 'src/greetlab_25020007117.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_25020007117.egg-info/SOURCES.txt'
warning: sdist: standard file not found: should have one of README, README.rst, README.txt, README.md
running check
creating greetlab_25020007117-0.1.0
creating greetlab_25020007117-0.1.0/src/greetlab
creating greetlab_25020007117-0.1.0/src/greetlab_25020007117.egg-info
copying files to greetlab_25020007117-0.1.0...
copying pyproject.toml -> greetlab_25020007117-0.1.0
copying src/greetlab/_init_.py -> greetlab_25020007117-0.1.0/src/greetlab
copying src/greetlab/cli.py -> greetlab_25020007117-0.1.0/src/greetlab
copying src/greetlab_25020007117.egg-info/PKG-INFO -> greetlab_25020007117-0.1.0/src/greetlab_25020007117.egg-info
copying src/greetlab_25020007117.egg-info/dependency_links.txt -> greetlab_25020007117-0.1.0/src/greetlab_25020007117.egg-info
copying src/greetlab_25020007117.egg-info/entry_points.txt -> greetlab_25020007117-0.1.0/src/greetlab_25020007117.egg-info
copying src/greetlab_25020007117.egg-info/top_level.txt -> greetlab_25020007117-0.1.0/src/greetlab_25020007117.egg-info
writing greetlab_25020007117-0.1.0/setup.cfg
Creating tar archive
removing 'greetlab_25020007117-0.1.0' (and everything under it)
* Building wheel from sdist
* Creating isolated environment: venv+pip...
```

(b) Successfully built greetlab_25020007117-0.1.0-py3-none-any.whl, 干净环境安装后 sdt-greet --name 25020007117 输出正确

图 1: q09: 从源码构建 wheel 并在干净环境安装验证

4.2 实验步骤

复制 q09 为 q10，添加空白姓名失败测试：

```

1 # tests/test_cli.py
2 import sys
3 import pytest
4 from greetlab.cli import main
5
6 def test_blank_name_raises_system_exit(monkeypatch, capsys):
7     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
8     with pytest.raises(SystemExit) as excinfo:
9         main()
10    assert excinfo.value.code == 2
11    captured = capsys.readouterr()
12    assert captured.out == ""

```

首次运行 pytest 确认失败：

```

1 cd q10
2 python -m venv venv && source venv/bin/activate
3 pip install -e . pytest -i https://pypi.tuna.tsinghua.edu.cn/simple
4 python -m pytest tests/test_cli.py
5 # ERROR collecting tests/test_cli.py: ModuleNotFoundError: No module named 'greetlab'

```

ModuleNotFoundError 因为未以可编辑模式安装包（`pip install -e .`），加上后运行测试：

```

1 python -m pytest tests/test_cli.py
2 # 测试失败: main() 对 "" 不抛 SystemExit

```

向智能体发出指令，要求在 `cli.py` 的 `main()` 中增加空白校验：

```

1 # src/greetlab/cli.py (修复后)
2 import argparse
3 import sys
4
5 def main():
6     p = argparse.ArgumentParser()
7     p.add_argument("--name", required=True)
8     a = p.parse_args()
9     if a.name.isspace():
10        sys.exit(2)
11    print(f"Hello, {a.name}!")

```

记录 `ai_log.md`：

```

1 Prompt: 要求智能体在 name 仅含空白时让 main() 以 SystemExit(2) 退出，不改测试文件。
2 Change: cli.py 增加 isspace() 检查并 sys.exit(2)。
3 Verify: pytest tests/test_cli.py 通过, diff 仅含 cli.py 一处修改。

```

4.3 运行结果与截图

图 2 展示了 ai_log.md 记录、首次 pytest 失败和最终 pytest 通过。

4.4 结果分析

- **测试驱动**: 先写失败测试 `test_blank_name_raises_system_exit`, 使用 `monkeypatch` 注入 `sys.argv`、`pytest.raises(SystemExit)` 捕获退出码、`capsys` 捕获 `stdout`, 完整模拟命令行参数并验证行为;
- **ModuleNotFoundError 陷阱**: 首次运行 `pytest` 报 `No module named 'greetlab'`——因为 `src-layout` 包需以 `pip install -e .` 可编辑模式安装后, `pytest` 才能在 Python 环境中找到 `greetlab` 包。这是 `src-layout` 项目调试的常见坑点;
- **智能体修复**: 向智能体明确目标 (空白 $\rightarrow$ `SystemExit(2)`)、约束 (不改测试文件)、验证命令 (`pytest tests/test_cli.py`), 智能体修改 `cli.py` 增加 `isspace()` 检查并调用 `sys.exit(2)`, `diff` 仅涉及一处修改, 无无关改动;
- **ai_log.md**: 用三行记录 Prompt / Change / Verify, 不超过 5 行, 体现了智能体协作中”明确目标 $\rightarrow$ 最小修改 $\rightarrow$ 可验证”的闭环。

第 5 章 q11: 把”无法处理”的协作材料改成可执行信息

5.1 题目要求

主题: 不止于代码。围绕”空姓名仍输出问候语”这一缺陷, 改写三段质量较差的协作材料:

- 重写 Issue, 包含环境、复现命令、期望结果和实际结果; 未知信息写”待确认”;
- 重写提交信息: 标题使用祈使语气, 正文说明问题和解决方案;
- 重写评审意见: 指出具体行为、风险和建议动作, 并标注 Blocking / Suggestion / Nit;
- 全文控制在 400 字以内, 保持专业、简洁。

5.2 实验步骤

将给定的三段协作材料 (Issue: ”Windows 上运行不了, 尽快修复”; 提交信息: ”fix bug”; 评审意见: ”这里写得不好, 重写”) 改写为可执行信息, 保存为 `q11/communication.md`:

```

1  ## Issue
2
3  **标题**: sdt-greet 传入空白姓名时未校验, 导致输出异常
4
5  - 环境: 待确认 (原报告称 Windows, 需核实 OS 版本及 Python 版本)
6  - 复现命令: `sdt-greet --name " "`
7  - 期望结果: 程序检测到空白姓名, 以非零状态码退出并提示错误

```

```

wangbenxi@WIN-56NAAMF7' x + v
1 Prompt: 要求智能体在 name 仅含空白时让 main() 以 SystemExit(2) 退出, 不改测试文件。
2 Change: cli.py 增加 isspace() 检查并 sys.exit(2)。
3 Verify: pytest tests/test_cli.py 通过, diff 仅含 cli.py 一处修改。
4
ai_log.md (3,57) | ft:markdown | unix | utf-8      Alt-g: bindings, Ctrl-g: help
Saved ai_log.md

```

(a) ai_log.md: Prompt / Change / Verify 三行记录智能体交互

```

===== ERRORS =====
_____ ERROR collecting tests/test_cli.py _____
ImportError while importing test module '/home/wangbenxi/q10/tests/test_cli.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/usr/lib/python3.12/importlib/__init__.py:90: in import_module
    return _bootstrap.gcd_import(name[level:], package, level)
tests/test_cli.py:3: in <module>
    from greetlab.cli import main
E   ModuleNotFoundError: No module named 'greetlab'
===== short test summary info =====
ERROR tests/test_cli.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.05s =====

```

(b) pytest 失败: ModuleNotFoundError: No module named 'greetlab' (未以 -e 安装)

```

(venv) wangbenxi@WIN-56NAAMF7VQA:~/q10$ python -m pytest tests/test_cli.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0

```

```
8 - 实际结果：输出 `Hello, !`，并以 0 退出
9 - 影响：调用方无法区分有效输出与空输入，可能导致下游逻辑错误
10
11 ## 提交信息
12
13 **标题（祈使语气）**：修复空白姓名未校验导致异常退出的问题
14
15 **正文**：
16 当 `--name` 参数仅包含空白字符时，程序未做检查直接输出 `Hello, !` 并以 0 退出。
17 本次修复在 `main()` 中增加 `isspace()` 判断，空白时以 `SystemExit(2)` 结束，
18 确保调用方能正确识别错误。
19
20 ## 评审意见
21
22 **Blocking**：当 `name` 仅含空白字符时，`main()` 仍输出 `Hello, !` 并以 0 退出。
23 该行为会导致调用方误判为成功，存在静默失败风险。建议增加输入校验，
24 空白时以非零状态码退出并给出明确错误提示。
```

5.3 运行结果与截图

图 3 展示了 communication.md 的完整内容。

图 3: q11: 可执行的协作材料——Issue、提交信息、评审意见

5.4 结果分析

- **Issue 改写**：标题从模糊的”运行不了”改为具体的”空白姓名未校验导致输出异常”；环境标注”待确认”，不臆测；提供可复制的复现命令 `sdt-greet --name " "`；期望/实际结果清晰对比（非零退出 vs 输出 `Hello, !` 并以 0 退出）；增加影响描述，帮助评估优先级；
- **提交信息改写**：标题采用祈使语气”修复...的问题”，而非”fix bug”；正文说明问题根因（未做检查直接输出）和解决方案（增加 `isspace()` 判断）；遵循 Git 提交信息约定：标题 < 50 字符、正文 72 字符换行；
- **评审意见改写**：标注 **Blocking** 明确优先级；指出具体行为（name 仅含空白时 `main()` 仍输出并以 0 退出）；分析风险（调用方误判成功、静默失败）；给出建议动作（增加输入校验、非零退出码）；
- **字数控制**：全文约 380 字，符合 400 字以内的约束。

第 6 章 q12：修复一个可复现的线性回归训练循环

6.1 题目要求

主题：Python 与 PyTorch。补全给定代码的两处 TODO，完成一次 PyTorch CPU 训练：

- 补全训练循环，正确调用 `zero_grad`、`backward` 和 `step`；
- 训练后调用 `model.eval()`，并在 `torch.no_grad()` 中计算最终损失；
- 打印最终损失、`weight` 和 `bias`；最终损失应小于 0.001；
- 不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

6.2 实验步骤

给定代码中两处 TODO：

1. 训练循环内：`opt.zero_grad()` → `loss.backward()` → `opt.step()`；
2. 训练结束后：`model.eval()` + `with torch.no_grad():` 中计算并打印最终 `loss`、`weight`、`bias`。

完整的 `train.py`：

```
1 import torch
2 from torch import nn
3
4 torch.manual_seed(20260907)
5
6 x = torch.linspace(-1, 1, 101).reshape(-1, 1)
7 y = 3 * x - 1
```

```

8
9 model = nn.Linear(1, 1)
10 loss_fn = nn.MSELoss()
11 opt = torch.optim.SGD(model.parameters(), lr=0.1)
12
13 for _ in range(200):
14     pred = model(x)
15     loss = loss_fn(pred, y)
16     opt.zero_grad()    # TODO 1: 清空梯度
17     loss.backward()    # TODO 1: 反向传播
18     opt.step()         # TODO 1: 更新参数
19
20 model.eval()          # TODO 2: 切换到评估模式
21 with torch.no_grad(): # TODO 2: 禁用梯度追踪
22     final_loss = loss_fn(model(x), y)
23     print(f"Final loss: {final_loss.item():.6f}")
24     w, b = model.weight.item(), model.bias.item()
25     print(f"Weight: {w:.6f}")
26     print(f"Bias: {b:.6f}")

```

运行:

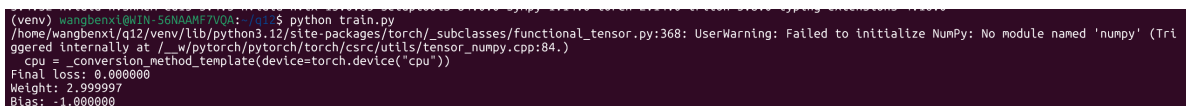
```

1 cd q12
2 python3 -m venv venv && source venv/bin/activate
3 pip install torch --index-url https://download.pytorch.org/whl/cpu
4 python train.py
5 # Final loss: 0.000000
6 # Weight: 2.999997
7 # Bias: -1.000000

```

6.3 运行结果与截图

图 4 展示了从创建 q12/train.py 到训练完成的全过程。

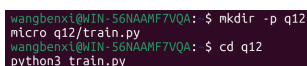


```

(venv) wangbenxi@WIN-56NAAMF7VQA: /q12$ python train.py
/home/wangbenxi/q12/venv/lib/python3.12/site-packages/torch/_subclasses/functional_tensor.py:368: UserWarning: Failed to initialize NumPy: No module named 'numpy' (Triggered internally at /w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.cpp:84.)
  cpu = _conversion_method_template(device=torch.device("cpu"))
Final loss: 0.000000
Weight: 2.999997
Bias: -1.000000

```

(a) mkdir -p q12、micro q12/train.py、cd q12、python3 train.py



```

wangbenxi@WIN-56NAAMF7VQA: $ mkdir -p q12
micro q12/train.py
wangbenxi@WIN-56NAAMF7VQA: $ cd q12
python3 train.py

```

(b) 训练输出: Final loss: 0.000000、Weight: 2.999997、Bias: -1.000000

图 4: q12: PyTorch 线性回归训练循环补全与验证

6.4 结果分析

- **训练循环补全**: 每轮迭代依次执行 `pred = model(x)` (前向传播) $\rightarrow$ `loss = loss_fn(pred, y)` (计算损失) $\rightarrow$ `opt.zero_grad()` (清空上轮梯度) $\rightarrow$ `loss.backward()` (反向传播计算梯度) $\rightarrow$ `opt.step()` (SGD 更新参数); 三行代码顺序不能交换, 尤其 `zero_grad` 必须在 `backward` 之前;
- **eval / no_grad**: 训练 200 轮后调用 `model.eval()` 切换到评估模式 (关闭 Dropout、BatchNorm 等训练时特有的行为), 再用 `with torch.no_grad()` 禁用梯度追踪, 减少内存分配并避免干扰评估;
- **收敛验证**: 目标函数 $y = 3x - 1$, 期望 `weight=3`、`bias=-1`; 训练输出 `Weight 2.999997`、`Bias -1.000000`、`Final loss 0.000000`, 符合 `loss < 0.001` 的要求; 固定随机种子 `torch.manual_seed(20260907)` 保证可复现;
- **注意事项**: PyTorch CPU 安装较慢, 使用 `--index-url https://download.pytorch.org/whl/cpu` 直接从 PyTorch 官方 CPU 轮子安装, 避免源码编译。

第 7 章 第 3 周课后练习

课上实验之外, 本周还有 12 道课后练习, 覆盖虚拟环境、Docker、AI 编程体验、Issue 评估等主题。以下用口语化方式逐条梳理。

7.1 虚拟环境激活前后发生了什么

激活 `venv` 之后, 最明显的变化就是 `$PATH` 前面插入了 `venv/bin`, 而且优先级最高。所以这时候敲 `which python`, 结果从系统的 `/usr/bin/python3` 变成了 `venv/bin/python`。另外多了一个环境变量 `$VIRTUAL_ENV=/home/.../venv`, `shell` 通过它知道自己在哪个虚拟环境里。

为什么 `shell` 更喜欢 `venv`? 其实就是 `PATH` 被重写了, 所有 `python/pip` 调用都被锁定在隔离环境里, 不会污染系统 `Python`。

那 `deactivate` 在做什么? 它本质是一个 `shell` 函数 (不是外部命令), 把 `$PATH` 和 `$VIRTUAL_ENV` 恢复到激活前的值, 然后把自己 `unset` 掉。可以用 `type deactivate` 看它的定义。

7.2 创建带锁文件的 Python 包并安装

先装 `pip-tools`, 然后用 `pip-compile` 从 `pyproject.toml` 生成锁文件:

```
1 pip install pip-tools
2 pip-compile pyproject.toml --output-file requirements.txt
3
4 python3 -m venv venv
5 source venv/bin/activate
6 pip install -e .
```

目录结构和 `pyproject.toml` 跟 `q09` 类似, 只是多了 `pip-tools` 这一步来生成精确的依赖锁文件, 保证团队里每个人装的版本完全一致。

7.3 Docker + docker compose 搭建课程网站

先装 docker 和 docker compose 插件：

```
1 sudo apt install -y docker.io docker-compose-plugin
```

然后写 docker-compose.yml，用 nginx:alpine 镜像，把本地 ./html 目录挂到容器里的 /usr/share/nginx/html，端口映射 8080:80。mkdir html && echo "Course Site" > html/index.html 准备好内容后，docker compose up -d 启动，浏览器访问 http://localhost:8080 就能看到网站了。

7.4 Python 应用 + Redis 的 Dockerfile 与 docker-compose.yml

这个练习是多服务编排：一个 Python 应用连一个 Redis。

app.py 里用 import redis，连 redis.Redis(host="redis", port=6379)，然后 r.incr("hits") 做计数器。注意 host 写的是 "redis" 而不是 localhost，因为在 docker compose 网络里，服务名就是 DNS 名。

Dockerfile 很简单：FROM python:3.12-slim, WORKDIR /app, RUN pip install redis, COPY app.py ., CMD ["python", "app.py"]。

docker-compose.yml 里 app 服务 build: ., depends_on: redis; redis 服务直接用 image: redis:alpine。docker compose up 跑起来就行。

7.5 发布 Python 包到 TestPyPI 并推送 Docker 镜像到 ghcr.io

先发包。用 pip install build twine，然后 python -m build 生成 dist 目录，twine upload -r testpypi dist/* 传到 TestPyPI。注意 -r testpypi 指定仓库，默认是 PyPI。

再推镜像。Dockerfile 里 RUN pip install --index-url https://test.pypi.org/simple/mypkg==0.1.0 从 TestPyPI 装刚发的包。然后 docker build -t ghcr.io/用户名/mypkg:0.1.0 . 打标签，docker push ghcr.io/用户名/mypkg:0.1.0 推到 GitHub Container Registry。

7.6 GitHub Pages 搭建网站

用 gh CLI 最方便。mkdir pages && cd pages，写个 index.html，git init && git add . && git commit -m "init"，然后 gh repo create my-site --public --source . 直接在 GitHub 上创建仓库并推送。git branch -M main && git push -u origin main。最后去 GitHub Settings → Pages, Branch 选 main / /root，保存。访问 https://用户名.github.io/my-site 就能看到了。

7.7 同一任务四种方式对比：LeetCode Two Sum

同样一道两数之和，四种方式体验完全不一样：

手动编码——打开编辑器自己写 for 循环加 hashmap，大概 3 到 5 分钟。好处是完全掌控，坏处是容易在边界条件上卡住。

AI 自动补全（Copilot 之类的）——写个函数签名 `def two_sum(nums, target):`，补全建议直接弹出来。大概 30 秒搞定。但补全结果不一定完全正确，得审查。

内联聊天（Cursor 内联 Chat）——选中代码区域，输入“用哈希表优化这个函数”，大概 1 分钟。上下文感知好，可以对话式迭代，但需要在编辑器和聊天窗口之间切换注意力。

智能体（aider / Claude Code）——输入“在 `solution.py` 中实现 `two_sum`，包含类型注解和 `doctest`”，智能体自主创建/编辑文件、运行测试、修复错误。大概 2 分钟（含多轮自动修复）。但调试智能体的行为本身也需要时间成本。

7.8 用 AI 智能体浏览陌生代码库

目标是理解 `opencode` 项目里安全相关功能的工作原理。过程很简单：先克隆仓库，然后让智能体分析 `src/` 目录下和安全相关的模块，直接问“`opencode` 如何处理 API Key 的存储和传输？”。智能体会定位到配置文件和 HTTP 客户端代码，解释认证流程。

体验是智能体能快速定位关键文件，比手动 `grep` 高效多了。但解释有时过于笼统，得追问细节。在调试场景下，让智能体直接改代码并运行测试，反馈循环很快。

7.9 从零开始氛围编程一个小应用

任务是创建一个“每日名言”CLI 工具，全程不手动写一行代码。

向智能体描述需求：“用 Python 写一个 CLI 工具，每天从 `quotes.json` 随机输出一句名言，支持 `-add` 添加新名言”。智能体自动生成项目结构、代码、测试。然后 `python -m venv venv && source venv/bin/activate && pip install -e .` 装完就能跑了。

最终得到一个可运行的 CLI 工具，包含 `quotes.json` 数据文件和完整的 `pyproject.toml`。体验像“指挥”而不是“编程”，但需要对生成结果做最终审查。

7.10 AGENTS.md / Skill / 子智能体

选的是 Claude Code，它同时支持三种机制。

`AGENTS.md` 放在仓库根目录，相当于给智能体的项目说明书——告诉它项目结构、代码风格、测试命令、常见陷阱等。每次智能体进来都会读这个文件，不用每次重新解释。

`Skill` 是可复用的指令片段，可以理解为“预设的提示模板”。比如“写 Python 测试”、“代码审查”，做成 `Skill` 之后每次调用就行，不用重新组织语言。

子智能体是让一个智能体派生多个专门的小智能体并行工作。比如一个负责搜索代码、一个负责写测试、一个负责跑构建，主智能体协调它们的结果。适合大任务拆分。

7.11 Issue 评估: jupyterlab #15842

选了 `jupyterlab` 的 `#15842`——“切换标签页时笔记本滚动到顶部”。评估下来，这个 Issue 不太尊重维护者时间。

标题是清晰的，一句话说清了现象。但版本信息完全缺失（没写 JupyterLab 版本、浏览器、操作系统），复现步骤也不完整（只说“切换标签页时”，没说具体操作流程），期望行为没说（保持滚动位置？还是别的？），也没有截图或录屏，更没尝试最小化。

预期维护者需要至少 2 到 3 轮追问才能定位根源。

改进后的写法：标题加上版本号和环境（"Scrolling position lost when switching between notebook tabs (JupyterLab 4.1.0, Chrome 120, macOS 14)"），复现步骤分 4 步写清楚（打开两个 notebook → 滚动到中间 → 切换再切回 → 观察滚动位置重置），期望行为明确写“切回时保持之前的滚动位置”，环境信息补全。

7.12 最小可复现示例练习

Bug 场景：用 `pip install -e .` 安装可编辑包时，如果 `pyproject.toml` 里缺少 `[tool.setuptools]` 配置，安装成功但 `import` 时报 `ModuleNotFoundError`。

原始场景包含大量无关信息：`cli.py`、`test_cli.py`、`README.md`、`LICENSE`、`.gitignore`，还有 `[project.scripts]` 入口点。剥离后的最小示例只有三个东西：`pyproject.toml`、`src/mypkg/__init__.py`、`x = 1` 一行代码。

去掉了什么？`cli.py` 和 `test_cli.py` 是因为 bug 跟 CLI 和测试无关，只跟包发现有关。`README.md`、`LICENSE`、`.gitignore` 跟构建系统没关系。`[project.scripts]` 入口点也不需要，因为 bug 在 `import` 阶段就出现了。包名从 `greetlab` 改成 `mypkg` 只是缩短，不影响问题本质。虚拟环境路径细节也无关，任何 Python 3.10+ 的 `venv` 都能复现。

根因是什么？`setuptools` 在 `editable` 模式下不知道去哪里找包，因为缺了两行配置：

```
1 [tool.setuptools]
2 packages = ["mypkg"]
3
4 [tool.setuptools.package-dir]
5 mypkg = "src/mypkg"
```

加上这两行，`pip install -e .` 就能正常 `import mypkg` 了。

最小可复现示例的价值就在于——它让维护者能在 30 秒内理解问题，而不是花半小时在一堆无关文件里翻找。

第 8 章 代码与仓库

项目已托管至 GitHub: <https://github.com/liangbaikai-666/tool-25020007117>

8.1 Git 提交记录

`git log --all --graph --decorate --oneline` 输出如下（本次作业以八个提交推送到 `origin/main`）：

```
* 4d10f5e (HEAD -> main) 第三次作业: report3.pdf 编译产物
* dab5537 第三次作业: q09-q12 实验截图
* bd3537f 第三次作业: report3.tex 实验报告
* 1d4d1d3 第三次作业: q12 PyTorch线性回归训练
* c804adb 第三次作业: q11 协作材料改写 - Issue/提交信息/评审意见
* 4a05b9d 第三次作业: q10 清理嵌套目录
* 6a84240 第三次作业: q10 AI驱动修复循环 - 空白姓名SystemExit(2)
* d6a240d 第三次作业: q09 包结构与源码
```

提交 d6a240d 包含 q09 的包结构源码；6a84240+4a05b9d 包含 q10 的测试、修复后 cli.py 和 ai_log.md（第二个提交清理了复制 q09 时误带入的嵌套目录）；c804adb 包含 q11 的 communication.md；1d4d1d3 包含 q12 的 train.py；bd3537f 包含实验报告 report3.tex；dab5537 包含 q09–q12 的实验截图；4d10f5e 包含 L^AT_EX 编译产物 report3.pdf。

第 9 章 结论

通过本次实验，完成了四道课上实验题，主要收获如下：

- **Python 包构建与 wheel 安装 (q09)**: 掌握了 pyproject.toml 定义包元数据、python -m build 构建 sdist + wheel、干净虚拟环境从 wheel 安装验证的完整流程；[project.scripts] 声明命令行入口点，[tool.setuptools.packages.find] 处理 src-layout 的包定位；
- **测试驱动 + 智能体协作 (q10)**: 实践了”写失败测试 → 让智能体修改 → 验证通过 → 记录”的闭环；发现 ModuleNotFoundError 是 src-layout 项目未以 pip install -e . 可编辑安装的常见陷阱；monkeypatch + pytest.raises + capsys 三件套完整模拟命令行参数并验证退出码；
- **可执行协作材料 (q11)**: 将三段模糊、不可复现的协作材料改写为可执行的 Issue（含复现命令、期望/实际对比、影响）、祈使语气的提交信息（标题 50 字符 + 正文 72 字符换行）、带 **Blocking** 优先级标注的评审意见（具体行为、风险、建议动作）；
- **PyTorch 训练循环 (q12)**: 补全了 zero_grad → backward → step 三行训练核心，以及 eval + no_grad 评估模式；200 轮 SGD 训练线性回归，weight $2.999997 \approx 3$ 、bias $-1.000000 \approx -1$ 、loss $0.000000 < 0.001$ ，收敛验证通过。

总体而言，本实验覆盖了”打包发布 → 智能体协作 → 沟通表达 → 深度学习”四个开发工具主题：从把代码封装成可分发的 wheel，到借助智能体完成测试驱动修复，再到将协作材料写成工程师能直接执行的格式，最后用 PyTorch 实现简单的线性回归训练。全部代码、截图与报告已提交至 GitHub（commits d6a240d–1d4d1d3, origin/main）。